**Hashtable<K,V>**

public class Hashtable<K,V> extends Dictionary<K,V> implements Map<K,V>, Cloneable, Serializable

* Legacy class
* Hahstable does not accept anything as a null as well as methods are synchronized
* HashMap accepts one null key and multiple null values, methods are not synchronized.
* Hashtable default capacity is 11 & load factor is 0.75

It is predefined class available in java.util package under Map interface from JDK 1.0.

Like Vector, Hashtable is also form the birth of java so called legacy class.

It is the sub class of Dictionary class which is an abstract class.

*The major difference between HashMap(1.2) and Hashtable(1.0) is, HashMap methods are not synchronized where as Hastable methods are synchronized.

HashMap can accept one null key and multiple null values where as Hashtable does not contain anything as a null(key and value both). if we try to add null then JVM will throw an exception i.e NullPointerException.

The initial default capacity of Hashtable class is 11 where as loadFactor is 0.75.

It has also same constructor as we have in HashMap.(4 constructors)

1) Hashtable hs1 = new Hashtable();
It will create the Hashtable Object with default capacity as 11 as well as load factor is 0.75

2) Hashtable hs2 = new Hashtable(int initialCapacity);
will create the Hashtable object with specified capacity

3) Hashtable hs3 = new Hashtable(int initialCapacity, float loadFactor);
we can specify our own initialCapacity and loadFactor

4) Hashtable hs = new Hashtable(Map c);
Interconversion of Map Collection

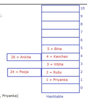
```
import java.util.*;
public class HashtableDemo
    {
     public static void main(String args[])
        {
          Hashtable<Integer,String> map = new Hashtable<>();
          map.put(1, "Java");
          map.put(2, "is");
          map.put(3, "best");
          map.put(4,"language");

          //map.put(5,null);  //NPE

          IO.println(map);

          IO.println("......................");

           for(Map.Entry m : map.entrySet())
              {
                 IO.println(m.getKey()+" = "+m.getValue());
              }
        }
}
```

How Hashtable<K,V> works internally ?

```
Hashtable<Integer,String> map = new Hashtable<>();
    map.put(1,"Priyanka");
    map.put(2,"Ruby");
    map.put(3,"Vibha");
    map.put(4,"Kanchan");

      map.putIfAbsent(5,"Bina");
      map.putIfAbsent(24,"Pooja");
      map.putIfAbsent(26,"Ankita");

    map.putIfAbsent(1,"Sneha");
    IO.println("Updated Map: "+map);
```

1 % 11 = 1

24 % 11 = 2




Output :

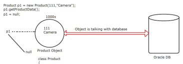
[Top to bottom & left and right]
[5 = Bina, 26=Ankita, 4=kanchan, Vibha, Pooja, Ruby, Priyanka]

```
import java.util.*;
public class HashtableDemo1
{
   public static void main(String args[])
     {
    Hashtable<Integer,String> map = new Hashtable<>();
     map.put(1,"Priyanka");
     map.put(2,"Ruby");
     map.put(3,"Vibha");
     map.put(4,"Kanchan");

       map.putIfAbsent(5,"Bina");
       map.putIfAbsent(24,"Pooja");
       map.putIfAbsent(26,"Ankita");

     map.putIfAbsent(1,"Sneha");
     IO.println("Updated Map: "+map);
  }
}
```

**protected void finalize() throws Throwable**

* It is a predefined non static method of Object class.

```
Product p1 = new Product(111,"Camera");
p1.getProductData();
p1 = null;
```




```
class Product
{
   public vodi finalize()
   {
       //Clean up activity OR Resourse Closing statements
   }
}
```

It is a predefined non static method of Object class.

Garbage Collector automatically call this method just before an object is eligible for garbage collection to perform clean-up activity.

Here clean-up activity **means closing the resources** associated with that object like file connection, database connection, network connection and so on we can say resource de-allocation.

Note :- a) GC calls finalize method only one per object.

b) This method is deprecated from java 9V.

c) Calling finalize() method on a particular object is an indication that Object is eligible for GC

```
package com.ravi.finalize_demo;

record Product(Integer id, String name)
{
    @Override
    public void finalize()
    {
        IO.println("Product object is eligible for GC");
    }
}

public class finalizeDemo
{
   public static void main(String[] args) throws InterruptedException
   {
     Product p1 = new Product(111, "Camera");
     IO.println(p1);

     p1=null;

     System.gc(); //Daemon Thread so life depends upon the user thread

     Thread.sleep(5000); //main thread will wait for 5 second

     IO.println(p1);

   }
}
```

**** What is the difference between final, finally and finalize()**

final :

It is used to provide some kind of restriction; by using final modifier we can declare our class, Method and Field/variable as a final.

finally:

If we open any resource as a part of **try block** then it should be compulsory closed inside finally block, [If We close the resources inside try block, It is always Risky] for resource handling purpose.

finalize() :

It is a method of Object class, we should override this method in the corresponding class to perform cleanup activity, GC will automatically call this method just before object destruction.

**WeakHashMap<K,V> [keys are very weak type]**

public class WeakHashMap<K,V> extends AbstractMap<K,V> implements Map<K,V>

It is a predefined class in java.util package under Map interface.It was introduced from JDK 1.2v onwards.

While working with HashMap, keys of HashMap are of strong reference type. This means the entry of map will not be deleted by the garbage collector even though the custom key is set to be null as well as Object is also not eligible for Garbage Collector.

On the other hand while working with WeakHashMap, keys of WeakHashMap are of weak reference type. This means the entry and corresponding object of a map is deleted by the garbage collector if the key value is set to be null/assigning a new object because it is of weak type.

So, HashMap dominates over Garbage Collector whereas Garbage Collector dominates over WeakHashMap.

It does not implements Cloneable and Serailizable because It is mainly used for inventory system where we need to manage the data and we can insert and delete object data frequently in the inventory.

It contains 4 types of Constructor :

1) WeakHashMap wm1 = new WeakHashMap();

Creates an empty WeakHashMap object with default capacity is 16 and load fator 0.75.

2) WeakHashMap wm2 = new WeakHashMap(int initialCapacity);

3) WeakHashMap wm3 = new WeakHashMap(int initialCapacity, float loadFactor);
Eg:- WeakHashMap wm = new WeakHashMap(10,0.9);

capacity - The capacity of this map is 10. Meaning, it can store 10 entries.

loadFactor - The load factor of this map is 0.9. This means whenever our hashtable is filled up by 90%, the entries are moved to a new hashtable of double the size of the original hashtable.

4) WeakHashMap wm4 = new WeakHashMap(Map m);

HashMap keys are strong so GC cannot delete the key object as well as entry of the HashMpa

WeakHashMap keys are weak type so GC will delete the object as well as map entry will also be deleted

```
package com.ravi.weak_hash_map;

import java.util.WeakHashMap;

record Product(Integer id, String name)
{
    @Override
    public void finalize()
    {
        IO.println("Product Object is eligible for GC");
    }
}

public class WeakHashMapDemo
{
    public static void main(String[] args) throws InterruptedException
    {
        Product p1 = new Product(555, "Laptop");
        WeakHashMap<Product,String> map = new WeakHashMap<>();
        map.put(p1, "Hyderabad");
        IO.println(map);

        p1 = null;
        System.gc();

        Thread.sleep(5000);

        IO.println(map); //{}

    }

}
```

**How to get system generated hashcode value :**

System class has provided a predefined native and static method called identityHashCode(Object obj), It is used to generate System hashcode.
It accepts Object as a parameter and return type of this method is int.

It is used to return default hash code of an object based on its memory identity, ignoring any overridden hashCode() method. [Object class hashcode]

public static native int identityHashCode(Object obj)

```
package com.ravi.hash;

class Customer
{
    private int id;
    private String name;

    public Customer(int id, String name) {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public int hashCode()          //If we commnet this overridden hashCode() method then
    {                              result will be same
        return 100;
    }

}
public class SystemHashCode
{
    public static void main(String[] args)
    {
        Customer c1 = new Customer(111, "Scott");
        IO.println(c1.hashCode());


        int hashCode = System.identityHashCode(c1);
        IO.println(hashCode);

    }
}
```